Adding debug chains for monitoring execution steps

Monitoring chain events allows us to track the execution flow and performance of their chains. This includes logging inputs and outputs at various stages, which is helpful for debugging and introspection purposes. The following listing shows the simplest way to add this to an existing chain.

Listing 8.5 Integrating debug information into the runnable chain

```
import { RunnableSequence } from "@langchain/core/runnables";
const debugChain = (input) => {
  console.log("Current Execution Context:", input);
  return input;
};

const finalChain = RunnableSequence.from([
  debugChain,
  chainGoogle,
]);

(async () => {
  await finalChain.invoke(
{ question: "What is artificial intelligence?" });
})();
```

This function is intended to log the current execution context for debugging purposes.

Add this chain to the sequence, which will log the input before passing it to the next runnable.

Invokes the finalChain with an object containing a question

Using the debug chain in your LangChain.js application provides a straightforward mechanism for logging the arguments or current context at specific points during the execution of your chain.

Note that the order of the operations matters here. Each runnable is executed in the sequence in which it is defined, which means that any changes made by earlier runnables will affect the inputs passed to subsequent ones.

Utilizing callback handlers

Callbacks provide valuable insights into the execution flow and can be customized further based on your application's requirements. These handlers allow us to hook into various stages of the application, enabling functionalities such as logging, monitoring, and error handling.

The following is an example of how to use the built-in `ConsoleCallbackHandler` in a LangChain.js application. This handler will log all calls made with this LLM:

```
import { ConsoleCallbackHandler } from "@langchain/core/tracers/console";
const handler = new ConsoleCallbackHandler();
const googleModel =
new ChatGoogleGenerativeAI({ apiKey, callbacks: [handler] } );
```

The `callbacks` option is basically a list of objects that has a predefined list of keys corresponding to an associated event type. Table 8.1 shows a short list of events together with their associated methods (as object property names).